

Advanced Java — Sessions 16 to 20

Course Flow

Session 16 → File I/O + Serialization



Session 17 → Multithreading



Session 18 → JDBC Introduction



Session 19 → JDBC Connection + Statement



Session 20 → PreparedStatement + CallableStatement + Transactions

SESSION 16 — File Input/Output & Serialization

1. Learning Objectives

By the end of this session, students should understand:

- Basic Java I/O
 - Input and Output
 - Streams
 - Byte streams
 - Character streams
 - File handling
 - Reading files
 - Writing files
 - Buffered streams
 - Serialization
 - Deserialization
-

2. What is I/O?

I/O = Input/Output

Input means receiving data.

Output means sending/displaying data.

Example:

Keyboard → Java Program → Screen

Another example:

File → Java Program → Screen

3. What is a Stream?

A stream represents a flow of data.

Source



Input Stream

↓
Java Program
↓
Output Stream
↓
Destination

Examples:

- Keyboard
- File
- Network
- Database

4. Types of Streams

Java I/O

```
|
|  └─ Byte Stream
|  |  └─ InputStream
|  |  └─ OutputStream
|  └─ Character Stream
|      └─ Reader
|      └─ Writer
```

Byte Stream

Used mainly for binary data:

- Images
- Audio
- Video
- PDF
- Binary files

Important classes:

FileInputStream

FileOutputStream

Character Stream

Used mainly for text.

Important classes:

FileReader

FileWriter

5. Writing to a File

```
import java.io.FileWriter;
import java.io.IOException;
```

```

public class FileWriteExample {

    public static void main(String[] args) {

        try {

            FileWriter writer =
                new FileWriter("student.txt");

            writer.write("Name: Amit\n");
            writer.write("Course: Advanced Java\n");
            writer.write("Marks: 85\n");

            writer.close();

            System.out.println(
                "Data written successfully."
            );

        } catch (IOException e) {

            System.out.println(
                "Error: " + e.getMessage()
            );

        }

    }
}

```

Output

Data written successfully.

File created:

student.txt

Contents:

Name: Amit

Course: Advanced Java

Marks: 85

6. Reading a File

```

import java.io.FileReader;
import java.io.IOException;

```

```

public class FileReadExample {

    public static void main(String[] args) {

        try {

            FileReader reader =
                new FileReader("student.txt");

            int ch;

            while ((ch = reader.read()) != -1) {

                System.out.print(
                    (char) ch
                );
            }

            reader.close();

        } catch (IOException e) {

            System.out.println(
                "Error: " + e.getMessage()
            );
        }
    }
}

```

Output

Name: Amit

Course: Advanced Java

Marks: 85

7. BufferedReader

Reading character by character is not always efficient.

BufferedReader allows convenient line-based reading.

```
import java.io.*;
```

```

public class BufferedReadExample {

    public static void main(String[] args) {

```

```
try {

    BufferedReader br =
        new BufferedReader(
            new FileReader(
                "student.txt"
            )
        );

    String line;

    while ((line = br.readLine()) != null) {

        System.out.println(line);
    }

    br.close();

} catch (IOException e) {

    e.printStackTrace();
}
}
```

8. BufferedWriter

```
import java.io.*;

public class BufferedWriteExample {

    public static void main(String[] args) {

        try {

            BufferedWriter bw =
                new BufferedWriter(
                    new FileWriter(
                        "employees.txt"
                    )
                );

        }
```

```

        bw.write("Employee 1");
        bw.newLine();

        bw.write("Employee 2");
        bw.newLine();

        bw.write("Employee 3");

        bw.close();

        System.out.println(
            "File created successfully."
        );

    } catch (IOException e) {

        e.printStackTrace();
    }
}
}

```

9. Byte Stream

FileInputStream

import java.io.*;

```

public class ByteReadExample {

    public static void main(String[] args) {

        try {

            FileInputStream fis =
                new FileInputStream(
                    "data.txt"
                );

            int data;

            while ((data = fis.read()) != -1) {

```

```
        System.out.print(
            (char) data
        );
    }

    fis.close();

} catch (IOException e) {

    e.printStackTrace();
}
}
```

10. FileOutputStream

```
import java.io.*;

public class ByteWriteExample {

    public static void main(String[] args) {

        try {

            FileOutputStream fos =
                new FileOutputStream(
                    "data.txt"
                );

            String message =
                "Welcome to Advanced Java";

            byte[] data =
                message.getBytes();

            fos.write(data);

            fos.close();

            System.out.println(
                "Data written."
            );
        }
    }
}
```

```
    } catch (IOException e) {  
  
        e.printStackTrace();  
    }  
}  
}
```

11. Serialization

Serialization means converting an object's state into a byte stream so it can be stored or transmitted.

Object



Serialization



Byte Stream



File

Deserialization:

File



Byte Stream



Object

12. Serializable Interface

```
import java.io.*;
```

```
class Student implements Serializable {
```

```
    private int id;  
    private String name;  
    private double marks;
```

```
    public Student(  
        int id,  
        String name,  
        double marks) {
```

```
        this.id = id;  
        this.name = name;
```

```
        this.marks = marks;
    }

    public void display() {

        System.out.println(
            "ID: " + id
        );

        System.out.println(
            "Name: " + name
        );

        System.out.println(
            "Marks: " + marks
        );
    }
}
```

13. Serialization Example

```
import java.io.*;

class Student implements Serializable {

    private int id;
    private String name;
    private double marks;

    public Student(
        int id,
        String name,
        double marks) {

        this.id = id;
        this.name = name;
        this.marks = marks;
    }

    public void display() {

        System.out.println(
```

```

        "ID: " + id
    );

    System.out.println(
        "Name: " + name
    );

    System.out.println(
        "Marks: " + marks
    );
}
}

public class SerializationDemo {

    public static void main(String[] args) {

        Student student =
            new Student(
                101,
                "Amit",
                88.5
            );

        try {

            ObjectOutputStream out =
                new ObjectOutputStream(
                    new FileOutputStream(
                        "student.ser"
                    )
                );

            out.writeObject(student);

            out.close();

            System.out.println(
                "Object serialized successfully."
            );

```

```

    } catch (IOException e) {

        e.printStackTrace();
    }
}

```

Output

Object serialized successfully.

14. Deserialization

```
import java.io.*;
```

```

public class DeserializationDemo {

    public static void main(String[] args) {

        try {

            ObjectInputStream in =
                new ObjectInputStream(
                    new FileInputStream(
                        "student.ser"
                    )
                );

            Student student =
                (Student) in.readObject();

            in.close();

            student.display();

        } catch (IOException |
            ClassNotFoundException e) {

            e.printStackTrace();
        }
    }
}

```

Output

ID: 101

Name: Amit

Marks: 88.5

Session 16 Assignment

Question

Create an Employee class containing:

- id
- name
- salary

Serialize the Employee object into:

employee.ser

Then deserialize and display it.

Solution

```
import java.io.*;
```

```
class Employee implements Serializable {
```

```
    private int id;
```

```
    private String name;
```

```
    private double salary;
```

```
    public Employee(
```

```
        int id,
```

```
        String name,
```

```
        double salary) {
```

```
        this.id = id;
```

```
        this.name = name;
```

```
        this.salary = salary;
```

```
    }
```

```
    public void display() {
```

```
        System.out.println(
```

```
            "ID: " + id
```

```
        );
```

```
        System.out.println(
```

```
            "Name: " + name
```

```
        );
```

```
        System.out.println(
            "Salary: " + salary
        );
    }
}
```

```
public class EmployeeSerialization {

    public static void main(String[] args) {

        Employee employee =
            new Employee(
                101,
                "Rahul",
                45000
            );

        try {

            ObjectOutputStream out =
                new ObjectOutputStream(
                    new FileOutputStream(
                        "employee.ser"
                    )
                );

            out.writeObject(employee);
            out.close();

            ObjectInputStream in =
                new ObjectInputStream(
                    new FileInputStream(
                        "employee.ser"
                    )
                );

            Employee e =
                (Employee) in.readObject();

            in.close();
        }
    }
}
```

```
        e.display();

    } catch (IOException |
            ClassNotFoundException ex) {

        ex.printStackTrace();
    }
}
```

Output

ID: 101

Name: Rahul

Salary: 45000.0

SESSION 17 — Multithreading

Learning Objectives

Students will understand:

- Process
 - Thread
 - Multithreading
 - Creating threads
 - Thread lifecycle
 - Thread scheduler
 - Thread priority
 - sleep()
 - join()
 - start()
 - run()
-

1. What is a Process?

A process is a running program.

Example:

Chrome

VS Code

MySQL

Java Program

Each can be a process.

2. What is a Thread?

A thread is a lightweight unit of execution inside a process.

Example:

Java Application

```
|  
├── Thread 1  
├── Thread 2  
└── Thread 3
```

3. What is Multithreading?

Executing multiple threads concurrently.

Example:

Thread 1 → Download

Thread 2 → Play Music

Thread 3 → Display UI

4. Creating Thread Using Thread Class

```
class MyThread extends Thread {
```

```
    @Override
```

```
    public void run() {
```

```
        for (int i = 1; i <= 5; i++) {
```

```
            System.out.println(
```

```
                "Thread: " + i
```

```
            );
```

```
        }
```

```
    }
```

```
}
```

```
public class ThreadDemo {
```

```
    public static void main(String[] args) {
```

```
        MyThread t =
```

```
            new MyThread();
```

```
        t.start();
```

```
    }
```

```
}
```

Output

Thread: 1

Thread: 2

Thread: 3

Thread: 4

Thread: 5

Important: start() vs run()

Students commonly make this mistake:

```
t.run();
```

This does not start a new thread.

Correct:

```
t.start();
```

start() causes the JVM to schedule the thread and invoke run().

5. Creating Thread Using Runnable

class MyTask implements Runnable {

```
    @Override
```

```
    public void run() {
```

```
        for (int i = 1; i <= 5; i++) {
```

```
            System.out.println(
```

```
                "Task: " + i
```

```
            );
```

```
        }
```

```
    }
```

```
}
```

```
public class RunnableDemo {
```

```
    public static void main(String[] args) {
```

```
        MyTask task =
```

```
            new MyTask();
```

```
        Thread t =
```

```
            new Thread(task);
```

```
        t.start();
```

```
    }
```

```
}
```

6. Multiple Threads

```
class MyThread extends Thread {  
  
    private String name;  
  
    public MyThread(String name) {  
  
        this.name = name;  
    }  
  
    @Override  
    public void run() {  
  
        for (int i = 1; i <= 5; i++) {  
  
            System.out.println(  
                name + " : " + i  
            );  
  
            try {  
  
                Thread.sleep(500);  
  
            } catch (InterruptedException e) {  
  
                Thread.currentThread()  
                    .interrupt();  
            }  
        }  
    }  
}  
  
public class MultipleThreadDemo {  
  
    public static void main(String[] args) {  
  
        MyThread t1 =  
            new MyThread("Thread A");  
  
        MyThread t2 =  
            new MyThread("Thread B");  
    }  
}
```

```
        t1.start();
        t2.start();
    }
}
```

Possible output

Thread A : 1

Thread B : 1

Thread A : 2

Thread B : 2

Thread B : 3

Thread A : 3

...

The exact order can vary.

7. Thread Lifecycle

Teach the conceptual lifecycle as:

NEW



RUNNABLE



RUNNING



BLOCKED / WAITING / TIMED WAITING



RUNNABLE



TERMINATED

Important states in Java's Thread.State:

NEW

RUNNABLE

BLOCKED

WAITING

TIMED_WAITING

TERMINATED

8. Thread Scheduler

The JVM/underlying runtime works with the operating system to schedule runnable threads.

Students should understand:

Thread execution order should not be assumed.

For example:

```
t1.start();
t2.start();
does not guarantee:
t1
t1
t1
t2
t2
```

9. Thread Priority

Java provides:

Thread.MIN_PRIORITY

Thread.NORM_PRIORITY

Thread.MAX_PRIORITY

Values:

MIN_PRIORITY = 1

NORM_PRIORITY = 5

MAX_PRIORITY = 10

Example:

```
class MyThread extends Thread {
```

```
    public void run() {
```

```
        System.out.println(
            getName() +
            " Priority: " +
            getPriority()
        );
```

```
    }
}
```

```
public class PriorityDemo {
```

```
    public static void main(String[] args) {
```

```
        MyThread t1 =
            new MyThread();
```

```
        MyThread t2 =
            new MyThread();
```

```

t1.setPriority(
    Thread.MIN_PRIORITY
);

t2.setPriority(
    Thread.MAX_PRIORITY
);

t1.start();
t2.start();
}
}

```

Important teaching point:

Priority is a scheduling hint, not a guarantee of execution order.

10. sleep()

Thread.sleep(1000);

Pauses the current thread for approximately one second.

11. join()

join() allows one thread to wait for another thread to finish.

class MyThread extends Thread {

```

    public void run() {

```

```

        for (int i = 1; i <= 3; i++) {

```

```

            System.out.println(i);

```

```

            try {

```

```

                Thread.sleep(500);

```

```

            } catch (InterruptedException e) {

```

```

                Thread.currentThread()

```

```

                    .interrupt();

```

```

            }

```

```

        }

```

```

    }

```

```

}

```

```

public class JoinDemo {

    public static void main(String[] args)
        throws InterruptedException {

        MyThread t =
            new MyThread();

        t.start();

        t.join();

        System.out.println(
            "Main thread finished."
        );
    }
}

```

Session 17 Assignment

Create two threads:

EvenThread

OddThread

Print even numbers from 2–10 and odd numbers from 1–9.

Solution

```

class EvenThread extends Thread {

    @Override
    public void run() {

        for (int i = 2; i <= 10; i += 2) {

            System.out.println(
                "Even: " + i
            );

            try {

                Thread.sleep(300);

            } catch (InterruptedException e) {

```

```

        Thread.currentThread()
            .interrupt();
    }
}
}
}

```

```

class OddThread extends Thread {

```

```

    @Override

```

```

    public void run() {

```

```

        for (int i = 1; i <= 9; i += 2) {

```

```

            System.out.println(
                "Odd: " + i
            );

```

```

        try {

```

```

            Thread.sleep(300);

```

```

        } catch (InterruptedException e) {

```

```

            Thread.currentThread()
                .interrupt();
        }
    }
}

```

```

}
}
}
}

```

```

public class NumberThreads {

```

```

    public static void main(String[] args) {

```

```

        EvenThread even =
            new EvenThread();

```

```

        OddThread odd =
            new OddThread();

```

```
        even.start();
        odd.start();
    }
}
```

SESSION 18 — JDBC Introduction

Learning Objectives

Students should understand:

- Database connectivity
 - JDBC
 - JDBC architecture
 - JDBC API
 - JDBC driver
 - Connection
 - Statement
 - ResultSet
 - Advantages of JDBC
-

1. What is JDBC?

JDBC = Java Database Connectivity

It allows Java applications to communicate with relational databases.

Java Application



JDBC



JDBC Driver



MySQL

2. JDBC Components

Important components:

Driver

Connection

Statement

PreparedStatement

CallableStatement

ResultSet

SQLException

3. JDBC Steps

Teach this as the standard JDBC workflow:

1. Add JDBC Driver



2. Establish Connection



3. Create Statement



4. Execute SQL



5. Process ResultSet



6. Close Resources

4. MySQL Database

Create:

```
CREATE DATABASE college;
```

```
USE college;
```

```
CREATE TABLE student (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    name VARCHAR(100),  
    email VARCHAR(100),  
    marks DOUBLE  
);
```

Insert sample data:

```
INSERT INTO student  
(name, email, marks)  
VALUES  
('Amit', 'amit@example.com', 85),  
('Rahul', 'rahul@example.com', 90);
```

5. Add MySQL Connector/J

For Maven, add the MySQL JDBC driver dependency to pom.xml.

Example:

```
<dependency>  
    <groupId>com.mysql</groupId>  
    <artifactId>mysql-connector-j</artifactId>  
    <version>9.4.0</version>  
</dependency>
```

For a classroom project, the exact driver version can be updated to the current version supported by your MySQL setup.

6. JDBC Connection

```
import java.sql.Connection;
import java.sql.DriverManager;

public class ConnectionDemo {

    public static void main(String[] args) {

        String url =
            "jdbc:mysql://localhost:3306/college";

        String username = "root";
        String password = "root";

        try {

            Connection connection =
                DriverManager.getConnection(
                    url,
                    username,
                    password
                );

            System.out.println(
                "Database connected successfully."
            );

            connection.close();

        } catch (Exception e) {

            e.printStackTrace();
        }
    }
}
```

Output

Database connected successfully.

7. JDBC Connection Utility

For larger applications, don't duplicate connection code.

```
import java.sql.Connection;
import java.sql.DriverManager;

public class DBConnection {

    private static final String URL =
        "jdbc:mysql://localhost:3306/college";

    private static final String USER =
        "root";

    private static final String PASSWORD =
        "root";

    public static Connection getConnection()
        throws Exception {

        return DriverManager.getConnection(
            URL,
            USER,
            PASSWORD
        );
    }
}
```

Session 18 Assignment

Write a Java program that connects to MySQL database college.

Solution

```
import java.sql.Connection;
import java.sql.DriverManager;

public class TestConnection {

    public static void main(String[] args) {

        try {

            Connection con =
                DriverManager.getConnection(
```

```

        "jdbc:mysql://localhost:3306/college",
        "root",
        "root"
    );

    System.out.println(
        "Connection successful!"
    );

    con.close();

} catch (Exception e) {

    e.printStackTrace();
}
}
}

```

SESSION 19 — Connection & Statement

Topics

- Connection interface
 - Statement interface
 - Execute SQL
 - executeQuery()
 - executeUpdate()
 - ResultSet
 - CRUD using Statement
-

1. Statement

Statement is used to execute SQL statements.

Statement statement =

```
connection.createStatement();
```

2. executeQuery()

Used primarily for queries returning a ResultSet, such as SELECT.

ResultSet rs =

```
statement.executeQuery(
    "SELECT * FROM student"
);
```

3. executeUpdate()

Used for operations such as:

INSERT

UPDATE

DELETE

Example:

```
int rows =
    statement.executeUpdate(
        "DELETE FROM student WHERE id=2"
    );
```

4. Select Example

```
import java.sql.*;
```

```
public class SelectDemo {
```

```
    public static void main(String[] args) {
```

```
        String url =
            "jdbc:mysql://localhost:3306/college";
```

```
        String user = "root";
        String password = "root";
```

```
        try (
            Connection con =
                DriverManager.getConnection(
                    url,
                    user,
                    password
                );
```

```
            Statement stmt =
                con.createStatement();
```

```
            ResultSet rs =
                stmt.executeQuery(
                    "SELECT * FROM student"
                )
        ) {
```

```
            while (rs.next()) {
```

```

        System.out.println(
            "ID: " +
            rs.getInt("id")
        );

        System.out.println(
            "Name: " +
            rs.getString("name")
        );

        System.out.println(
            "Email: " +
            rs.getString("email")
        );

        System.out.println(
            "Marks: " +
            rs.getDouble("marks")
        );

        System.out.println(
            "-----"
        );
    }

} catch (SQLException e) {

    e.printStackTrace();
}
}
}

```

Output

```

ID: 1
Name: Amit
Email: amit@example.com
Marks: 85.0
-----
ID: 2
Name: Rahul

```

Email: rahul@example.com

Marks: 90.0

5. Insert Using Statement

```
import java.sql.*;
```

```
public class InsertDemo {
```

```
    public static void main(String[] args) {
```

```
        String url =
```

```
            "jdbc:mysql://localhost:3306/college";
```

```
        try (
```

```
            Connection con =
```

```
                DriverManager.getConnection(
```

```
                    url,
```

```
                    "root",
```

```
                    "root"
```

```
                );
```

```
            Statement stmt =
```

```
                con.createStatement()
```

```
        ) {
```

```
            String sql =
```

```
                "INSERT INTO student " +
```

```
                "(name,email,marks) " +
```

```
                "VALUES " +
```

```
                "('Priya'," +
```

```
                "'priya@example.com'," +
```

```
                "92)";
```

```
            int rows =
```

```
                stmt.executeUpdate(sql);
```

```
            System.out.println(
```

```
                rows +
```

```
                " record inserted."
```

```
            );
```

```
    } catch (SQLException e) {  
  
        e.printStackTrace();  
    }  
}  
}
```

6. Update

```
String sql =  
    "UPDATE student " +  
    "SET marks=95 " +  
    "WHERE id=1";  
  
int rows =  
    stmt.executeUpdate(sql);  
  
System.out.println(  
    rows + " record updated."  
);
```

7. Delete

```
String sql =  
    "DELETE FROM student WHERE id=2";  
  
int rows =  
    stmt.executeUpdate(sql);  
  
System.out.println(  
    rows + " record deleted."  
);
```

Statement Problem

Suppose user enters:

name = Amit

and the application creates:

```
SELECT * FROM student WHERE name='Amit'
```

Building SQL by concatenating untrusted input can create SQL injection vulnerabilities.

Therefore:

Use PreparedStatement for user-supplied values.

This leads directly into Session 20.

Session 19 Assignment

Create a Java program to display students having marks greater than 80.

Solution

```
import java.sql.*;

public class MarksDemo {

    public static void main(String[] args) {

        String url =
            "jdbc:mysql://localhost:3306/college";

        try (
            Connection con =
                DriverManager.getConnection(
                    url,
                    "root",
                    "root"
                );

            Statement stmt =
                con.createStatement();

            ResultSet rs =
                stmt.executeQuery(
                    "SELECT * FROM student " +
                    "WHERE marks > 80"
                )
        ) {

            while (rs.next()) {

                System.out.println(
                    rs.getString("name") +
                    " - " +
                    rs.getDouble("marks")
                );
            }

        } catch (SQLException e) {
```

```
        e.printStackTrace();
    }
}
}
```

SESSION 20 — PreparedStatement, CallableStatement & Transactions

This is the most important JDBC session of these three.

1. PreparedStatement

PreparedStatement allows SQL statements with parameters.

Example:

```
SELECT * FROM student
```

```
WHERE id = ?
```

The ? is a parameter placeholder.

2. Why PreparedStatement?

Advantages:

- Prevents SQL injection when used correctly
 - Cleaner parameter handling
 - Better for repeated statements
 - Easier to read
 - Supports typed parameters
-

3. Insert Using PreparedStatement

```
import java.sql.*;
```

```
public class PreparedInsert {
```

```
    public static void main(String[] args) {
```

```
        String url =
            "jdbc:mysql://localhost:3306/college";
```

```
        String sql =
            "INSERT INTO student " +
            "(name,email,marks) " +
            "VALUES (?, ?, ?)";
```

```
        try (
            Connection con =
```

```

        DriverManager.getConnection(
            url,
            "root",
            "root"
        );

        PreparedStatement ps =
            con.prepareStatement(sql)
    ) {

        ps.setString(
            1,
            "Neha"
        );

        ps.setString(
            2,
            "neha@example.com"
        );

        ps.setDouble(
            3,
            91.5
        );

        int rows =
            ps.executeUpdate();

        System.out.println(
            rows +
            " record inserted."
        );

    } catch (SQLException e) {

        e.printStackTrace();
    }
}
}

```

4. Select Using PreparedStatement

```

import java.sql.*;

public class PreparedSelect {

    public static void main(String[] args) {

        String url =
            "jdbc:mysql://localhost:3306/college";

        String sql =
            "SELECT * FROM student " +
            "WHERE marks > ?";

        try (
            Connection con =
                DriverManager.getConnection(
                    url,
                    "root",
                    "root"
                );

            PreparedStatement ps =
                con.prepareStatement(sql)
        ) {

            ps.setDouble(1, 80);

            try (
                ResultSet rs =
                    ps.executeQuery()
            ) {

                while (rs.next()) {

                    System.out.println(
                        rs.getString("name")
                        + " - "
                        + rs.getDouble(
                            "marks"
                        )
                    );
                }
            }
        }
    }
}

```

```
        }  
    }  
  
    } catch (SQLException e) {  
  
        e.printStackTrace();  
    }  
}  
}
```

5. Update

String sql =

```
"UPDATE student " +  
"SET marks=? " +  
"WHERE id=?";
```

```
PreparedStatement ps =  
    con.prepareStatement(sql);
```

```
ps.setDouble(1, 95);  
ps.setInt(2, 1);
```

```
ps.executeUpdate();
```

6. Delete

String sql =

```
"DELETE FROM student WHERE id=?";
```

```
PreparedStatement ps =  
    con.prepareStatement(sql);
```

```
ps.setInt(1, 3);
```

```
ps.executeUpdate();
```

7. CallableStatement

CallableStatement is used to call stored procedures.

Architecture:

Java



CallableStatement



Stored Procedure



MySQL

Create Stored Procedure

DELIMITER //

```
CREATE PROCEDURE getStudentById(  
    IN studentId INT  
)  
BEGIN
```

```
    SELECT *  
    FROM student  
    WHERE id = studentId;
```

```
END //
```

```
DELIMITER ;
```

Java Code

```
import java.sql.*;
```

```
public class CallableDemo {
```

```
    public static void main(String[] args) {
```

```
        String url =  
            "jdbc:mysql://localhost:3306/college";
```

```
        String sql =  
            "{CALL getStudentById(?)}";
```

```
        try (  
            Connection con =  
                DriverManager.getConnection(  
                    url,  
                    "root",  
                    "root"  
                );
```

```

        CallableStatement cs =
            con.prepareCall(sql)
    ) {

        cs.setInt(1, 1);

        try (
            ResultSet rs =
                cs.executeQuery()
        ) {

            while (rs.next()) {

                System.out.println(
                    "ID: " +
                    rs.getInt("id")
                );

                System.out.println(
                    "Name: " +
                    rs.getString("name")
                );

                System.out.println(
                    "Email: " +
                    rs.getString("email")
                );
            }
        }

    } catch (SQLException e) {

        e.printStackTrace();
    }
}

```

8. Transaction Management

A transaction is a group of database operations treated as one logical unit.

Example:

Bank Transfer

Account A

↓

Withdraw ₹500

↓

Account B

↓

Deposit ₹500

Both should succeed, or neither should take effect.

Transaction Properties

Introduce ACID:

A → Atomicity

C → Consistency

I → Isolation

D → Durability

9. JDBC Transaction

By default, JDBC connections generally operate with auto-commit enabled.

To manually control a transaction:

```
con.setAutoCommit(false);
```

Then:

```
con.commit();
```

or:

```
con.rollback();
```

10. Bank Transfer Example

Database:

```
CREATE TABLE account (
```

```
    id INT PRIMARY KEY,
```

```
    name VARCHAR(100),
```

```
    balance DOUBLE
```

```
);
```

Sample:

```
INSERT INTO account
```

```
VALUES
```

```
(1, 'Amit', 10000),  
(2, 'Rahul', 5000);
```

Complete Transaction Code

```
import java.sql.*;
```

```
public class TransactionDemo {  
  
    public static void main(String[] args) {  
  
        String url =  
            "jdbc:mysql://localhost:3306/college";  
  
        String debitSQL =  
            "UPDATE account " +  
            "SET balance = balance - ? " +  
            "WHERE id = ?";  
  
        String creditSQL =  
            "UPDATE account " +  
            "SET balance = balance + ? " +  
            "WHERE id = ?";  
  
        try (  
            Connection con =  
                DriverManager.getConnection(  
                    url,  
                    "root",  
                    "root"  
                );  
  
            PreparedStatement debit =  
                con.prepareStatement(  
                    debitSQL  
                );  
  
            PreparedStatement credit =  
                con.prepareStatement(  
                    creditSQL  
                )  
        ) {
```

```
con.setAutoCommit(false);

double amount = 1000;

// Debit
debit.setDouble(1, amount);
debit.setInt(2, 1);

int debitRows =
    debit.executeUpdate();

// Credit
credit.setDouble(1, amount);
credit.setInt(2, 2);

int creditRows =
    credit.executeUpdate();

if (debitRows == 1 &&
    creditRows == 1) {

    con.commit();

    System.out.println(
        "Transaction successful."
    );

} else {

    con.rollback();

    System.out.println(
        "Transaction rolled back."
    );
}

} catch (SQLException e) {

    e.printStackTrace();
}
```

```
}  
}
```

11. Rollback Example

```
try {  
  
    con.setAutoCommit(false);  
  
    // Operation 1  
    // Operation 2  
    // Operation 3  
  
    con.commit();  
  
} catch (SQLException e) {  
  
    try {  
  
        con.rollback();  
  
    } catch (SQLException rollbackError) {  
  
        rollbackError.printStackTrace();  
    }  
}
```

Session 20 Assignment 1 — PreparedStatement CRUD

Create a Java program that performs:

1. Insert Student
2. Update Student
3. Search Student
4. Delete Student

using PreparedStatement.

Solution

```
import java.sql.*;
```

```
public class StudentCRUD {
```

```
    static final String URL =  
        "jdbc:mysql://localhost:3306/college";
```

```

static final String USER =
    "root";

static final String PASSWORD =
    "root";

public static void main(String[] args) {

    insertStudent(
        "Karan",
        "karan@example.com",
        87
    );

    updateStudent(1, 95);

    searchStudent(1);

    deleteStudent(3);
}

static void insertStudent(
    String name,
    String email,
    double marks) {

    String sql =
        "INSERT INTO student " +
        "(name,email,marks) " +
        "VALUES (?, ?, ?)";

    try (
        Connection con =
            DriverManager.getConnection(
                URL,
                USER,
                PASSWORD
            );

        PreparedStatement ps =
            con.prepareStatement(sql)

```

```

    ){

        ps.setString(1, name);
        ps.setString(2, email);
        ps.setDouble(3, marks);

        ps.executeUpdate();

        System.out.println(
            "Student inserted."
        );

    } catch (SQLException e) {

        e.printStackTrace();
    }
}

static void updateStudent(
    int id,
    double marks) {

    String sql =
        "UPDATE student " +
        "SET marks=? " +
        "WHERE id=?";

    try (
        Connection con =
            DriverManager.getConnection(
                URL,
                USER,
                PASSWORD
            );

        PreparedStatement ps =
            con.prepareStatement(sql)
    ){

        ps.setDouble(1, marks);
        ps.setInt(2, id);
    }
}

```

```

        ps.executeUpdate();

        System.out.println(
            "Student updated."
        );
    } catch (SQLException e) {

        e.printStackTrace();
    }
}

static void searchStudent(int id) {

    String sql =
        "SELECT * FROM student " +
        "WHERE id=?";

    try (
        Connection con =
            DriverManager.getConnection(
                URL,
                USER,
                PASSWORD
            );

        PreparedStatement ps =
            con.prepareStatement(sql)
    ) {

        ps.setInt(1, id);

        try (
            ResultSet rs =
                ps.executeQuery()
        ) {

            if (rs.next()) {

                System.out.println(

```

```

        "ID: " +
        rs.getInt("id")
    );

    System.out.println(
        "Name: " +
        rs.getString("name")
    );

    System.out.println(
        "Email: " +
        rs.getString("email")
    );

    System.out.println(
        "Marks: " +
        rs.getDouble("marks")
    );

    } else {

        System.out.println(
            "Student not found."
        );
    }
}

} catch (SQLException e) {

    e.printStackTrace();
}
}

static void deleteStudent(int id) {

    String sql =
        "DELETE FROM student " +
        "WHERE id=?";

    try (
        Connection con =

```

```

        DriverManager.getConnection(
            URL,
            USER,
            PASSWORD
        );

        PreparedStatement ps =
            con.prepareStatement(sql)
    ) {

        ps.setInt(1, id);

        ps.executeUpdate();

        System.out.println(
            "Student deleted."
        );

    } catch (SQLException e) {

        e.printStackTrace();
    }
}
}

```

Session 20 Assignment 2 — Final JDBC Mini Project

Student Management System

Students should create a console application:

=====

STUDENT MANAGEMENT

=====

1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

Enter choice:

Required Technologies

Java



JDBC



PreparedStatement



MySQL

Database

CREATE DATABASE college;

USE college;

CREATE TABLE student (

id INT PRIMARY KEY AUTO_INCREMENT,

name VARCHAR(100) NOT NULL,

email VARCHAR(150),

marks DOUBLE

);

Suggested Class Structure

For this final assignment, teach students to move beyond one huge Java file:

StudentManagement

|

├── DBConnection.java

|

├── Student.java

|

├── StudentDAO.java

|

└── Main.java

Architecture:

Main



StudentDAO



DBConnection

↓

JDBC

↓

MySQL

This will also prepare them for your later Servlet → DAO → JDBC → MySQL and Spring Boot → Service → Repository → JPA → MySQL architecture.

Teaching Plan by Time

A practical 2–2.5 hour lecture can be structured as follows:

Session	Theory	Live Coding	Student Practice	Assignment
16 — File I/O	35 min	45 min	25 min	20 min
17 — Multithreading	35 min	45 min	25 min	20 min
18 — JDBC Intro	30 min	45 min	30 min	20 min
19 — Statement	30 min	50 min	30 min	20 min
20 — Prepared/Callable/Transaction	40 min	60 min	30 min	30 min

Important Viva Questions

Session 16

1. What is I/O?
 2. What is a stream?
 3. Difference between byte and character streams?
 4. What is FileReader?
 5. What is BufferedReader?
 6. What is serialization?
 7. What is deserialization?
 8. What is Serializable?
 9. Why is transient used?
-

Session 17

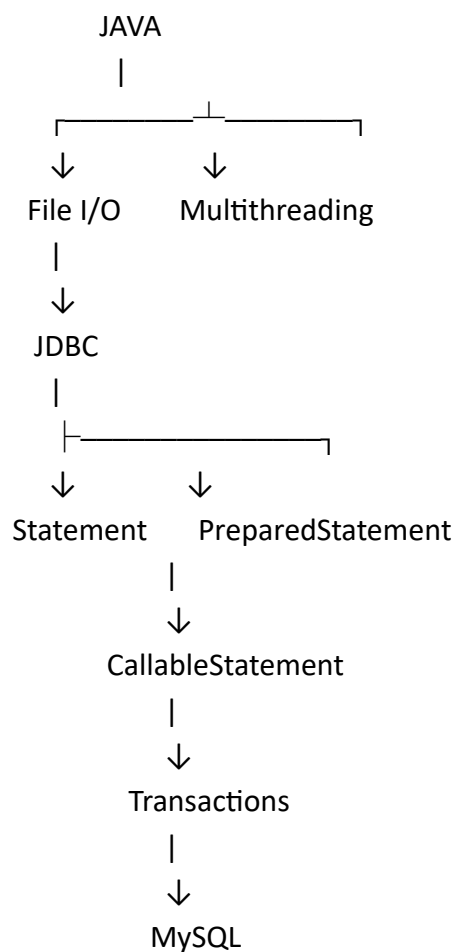
1. What is a thread?
 2. What is multithreading?
 3. Difference between process and thread?
 4. How can you create a thread?
 5. Difference between start() and run()?
 6. What is thread lifecycle?
 7. What is thread scheduler?
 8. What is thread priority?
 9. What does sleep() do?
 10. What does join() do?
-

Sessions 18–20

1. What is JDBC?
 2. What are JDBC components?
 3. What is JDBC Driver?
 4. What is Connection?
 5. What is Statement?
 6. Difference between executeQuery() and executeUpdate()?
 7. What is ResultSet?
 8. What is PreparedStatement?
 9. Why is PreparedStatement preferred for user input?
 10. What is CallableStatement?
 11. What is a stored procedure?
 12. What is a transaction?
 13. What is commit()?
 14. What is rollback()?
 15. Explain ACID properties.
 16. What is auto-commit?
-

Final Practical Outcome

After Session 20, students should be able to build:



This sequence is particularly useful because it sets up the next stage of your Advanced Java course very naturally:

Core Java



File I/O



Multithreading



JDBC



Servlet



JSP



Hibernate/JPA



Spring



Spring Boot



REST API



Spring Data JPA



MySQL



Final Project

One recommendation: when you teach JDBC, make PreparedStatement + DAO + transactions the practical focus, rather than spending too much time on Statement. Statement is important for understanding the JDBC API, but PreparedStatement is the pattern students should actually use in applications.